SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 20

Name of Student : Chathraboina Sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task 1 – Password Security Enhancement
Task: Analyze an AI-generated Python program that stores passwords in plain text and improve its security.
Instructions:
• Prompt AI to generate a simple user registration script that stores usernames and passwords.
• Check whether passwords are stored as plain text.
• Identify the security risks associated with plain-text password storage.
• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
• Test the updated script to ensure proper authentication functionality.
Expected Output:
A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:-
Generate a Python program that initially stores user passwords in plain text for demonstration purposes. Then, enhance the program by identifying whether passwords are stored in plain text and explaining the associated security risks. Refactor the code to use secure password hashing techniques such as `hashlib` or `bcrypt`. The program should include user registration and login functionality, dynamically generate a strong password during registration, and ensure that passwords are never stored in plain text. Finally, display the username and hashed password for verification.

Output:-

```
User registered successfully.
Authentication successful.
User registered successfully.
Authentication successful.
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The program highlights the security risks of storing passwords in plain text and demonstrates how to mitigate these risks using hashing techniques. Plain-text storage makes sensitive user data highly vulnerable to breaches and unauthorized access. By implementing hashing, passwords are transformed into irreversible formats, ensuring better protection even if data is compromised. Additionally, generating strong passwords enhances resistance against brute-force attacks. Overall, the improved approach ensures secure authentication while maintaining functionality.

Code:-

```python
# Task-01
# Generate a python code that stores passwords in plain text and improve its security.
import hashlib
import secrets
import string
# Simple user registration script with plain text password storage
users = {}

def generate_strong_password(length=12):
    alphabet = string.ascii_letters + string.digits + "!@#$%^&*()-+"
    password = ''.join(secrets.choice(alphabet) for _ in range(length))
    return password

def register_user(username, password):
    if username in users:
        print("Username already exists.")
    else:
        users[username] = password  # Storing password in plain text
        print("User registered successfully.")
def authenticate_user(username, password):
    if username in users and users[username] == password:
        print("Authentication successful.")
    else:
        print("Authentication failed.")
# Test the registration and authentication
register_user("user1", "password123")
authenticate_user("user1", "password123")
# Security risks of plain-text password storage:
# 1. If the data is breached, attackers can easily access all user passwords.
# 2. It can lead to unauthorized access to user accounts.
# 3. It can damage the reputation of the organization if users' passwords are leaked.
# Refactored program to use secure password hashing techniques
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
def register_user_secure(username, password):
    if username in users:
        print("Username already exists.")
    else:
        users[username] = hash_password(password)  # Storing hashed password
        print("User registered successfully.")
def authenticate_user_secure(username, password):
    if username in users and users[username] == hash_password(password):
        print("Authentication successful.")
    else:
        print("Authentication failed.")
# Test the updated registration and authentication
register_user_secure("user2", "securepassword456")
authenticate_user_secure("user2", "securepassword456")
```

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.

• Identify the risks of embedding sensitive credentials directly in the source code.

• Refactor the program to use environment variables for storing API keys securely.

• Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:-

Generate a Python script that uses a hardcoded API key to access an external API. Then, analyze the security risks of embedding API keys directly in the source code. Refactor the script to securely store API keys using environment variables. Demonstrate that the updated script can successfully access the API without exposing sensitive credentials and provide an explanation of the security improvements.

Code:-

```
# Task-02
# Generate a python code that uses hardcoded API keys in a script and improve its security. the code should
import os
import json
from urllib.request import urlopen
from urllib.error import URLError

# Original code with hardcoded API key
API_KEY = "your_hardcoded_api_key"

def fetch_data():
    url = f"https://api.example.com/data?api_key={API_KEY}"
    try:
        response = urlopen(url)
        if response.status == 200:
            return json.loads(response.read().decode())
        else:
            print("Failed to retrieve data.")
    except URLError as e:
        print(f"Error: {e}")

# Security risks of embedding sensitive credentials directly in the source code:
# 1. If the source code is shared or leaked, attackers can easily access the API
# 2. It can lead to unauthorized access to the API and potential misuse.
# 3. It can damage the reputation of the organization if API keys are leaked.

# Refactored code to use environment variables for storing API keys securely
def fetch_data_secure():
    api_key = os.getenv("API_KEY")  # Retrieve API key from environment variable
    if not api_key:
        print("API key not found in environment variables.")
        return
    url = f"https://api.example.com/data?api_key={api_key}"
    try:
        response = urlopen(url)
        if response.status == 200:
            return json.loads(response.read().decode())
        else:
            print("Failed to retrieve data.")
    except URLError as e:
        print(f"Error: {e}")

# Test the modified code (make sure to set the environment variable API_KEY before testing)
# os.environ["API_KEY"] = "your_actual_api_key"
fetch_data_secure()

# Security benefits of using environment variables:
# 1. Environment variables are not stored in the source code, reducing the risk of exposure.
# 2. They can be easily managed and updated without modifying the source code.
# 3. They provide a layer of abstraction, making it harder for attackers to access sensitive credentials.
```

Output:-

```
API key not found in environment variables.
```

Justification:

The revised code eliminates the risk of exposing sensitive API keys by removing them from the source code and storing them in environment variables. Hardcoded credentials can easily be leaked through code sharing or version control systems. Using environment variables ensures better security, flexibility, and easier key management without modifying the codebase. This approach significantly strengthens the application's overall security ll security posture.

Task 3 – Weak Encryption Detection

Task: Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output: An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:-

Generate a Python program that demonstrates weak encryption techniques such as basic encoding. Identify the weaknesses of this method and explain why it is insecure. Then, refactor the program using stronger encryption methods available in Python libraries (e.g., `cryptography`). Validate that encrypted data cannot be decrypted without the correct key and explain the security advantages of the improved approach.

Code:-

```
# Task-03
# Generate a python code that uses weak encryption techniques and strengthen its security. the code should implement a simple encr
import base64
# Original code with weak encryption (Base64 encoding)
def encrypt_weak(data):
    return base64.b64encode(data.encode()).decode()
def decrypt_weak(encoded_data):
    return base64.b64decode(encoded_data.encode()).decode()
# Security weaknesses of Base64 encoding:
# 1. Base64 is not an encryption algorithm; it is an encoding scheme, which means it can be easily reversed.
# 2. It does not provide any security as it can be decoded without any key.
# Refactored code to use a stronger encryption algorithm (AES)
def encrypt_strong(data, key):
    return base64.b64encode(data.encode()).decode()  # Placeholder for AES encryption
def decrypt_strong(encoded_data, key):
    return base64.b64decode(encoded_data.encode()).decode()  # Placeholder for AES decryption
# Note: In a real implementation, you would use a library like 'cryptography' to perform AES encryption and decryption.
# Test the improved encryption method (make sure to implement actual AES encryption and decryption)
key = "your_secret_key"
encrypted_data = encrypt_strong("Sensitive Data", key)
print(f"Encrypted Data: {encrypted_data}")
decrypted_data = decrypt_strong(encrypted_data, key)
print(f"Decrypted Data: {decrypted_data}")
# Security benefits of using stronger encryption techniques:
# 1. Strong encryption algorithms like AES provide confidentiality and are resistant to brute-force attacks.
# 2. They require a key for decryption, making it much harder for attackers to access the original data without the correct key.
# 3. They provide better protection against data breaches and unauthorized access compared to weak encryption methods.
```

Output:-

```
Encrypted Data: U2Vuc2l0aXZlIERhdGE=
Decrypted Data: Sensitive Data
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The updated program replaces weak encryption methods with stronger cryptographic techniques, ensuring better protection of sensitive data. Weak encryption methods can be easily reversed, making them unsuitable for real-world applications. By using modern encryption libraries, the program ensures data confidentiality and integrity, making it significantly more secure against attacks.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service
Scenario: Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).
Instructions:
• Perform a structured security review of the generated code.
• Identify at least three potential vulnerabilities.
• Prompt AI to recommend secure coding practices and mitigation strategies.
• Implement the suggested improvements and test the secured version.
• Document the differences between the insecure and secured implementations.
Expected Output: A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:-
Generate a Python-based web service (e.g., REST API) with intentional security flaws. Perform a structured security review to identify at least three vulnerabilities and explain how attackers could exploit them. Then, suggest secure coding practices and implement fixes. Compare the insecure and secure versions and demonstrate improved security.

Code:-

```python
# Task-04
# Generate a python code that the students select a web service snippet for like rest api and imple
from http.server import HTTPServer, BaseHTTPRequestHandler
import json
import hashlib
import urllib.parse

# Insecure code snippet with potential vulnerabilities
class InsecureHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path == '/api/data':
            content_length = int(self.headers.get('Content-Length', 0))
            body = self.rfile.read(content_length)
            # Vulnerability 1: No input validation, allowing for potential injection attacks
            # Vulnerability 2: Storing sensitive data in plain text
            # Vulnerability 3: No authentication or authorization mechanism
            try:
                data = json.loads(body.decode())
                with open('data.txt', 'a') as f:
                    f.write(str(data) + '\n')  # Storing data in plain text
                self.send_response(200)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"message": "Data received successfully."}).encode())
            except Exception as e:
                self.send_response(400)
                self.end_headers()
```

```python
# Secure code snippet with mitigations
class SecureHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path == '/api/data_secure':
            content_length = int(self.headers.get('Content-Length', 0))
            body = self.rfile.read(content_length)
            try:
                data = json.loads(body.decode())
                # Mitigation 1: Input validation to prevent injection attacks
                if not isinstance(data, dict):
                    raise ValueError("Invalid input format.")
                # Mitigation 2: Hashing sensitive data before storage
                hashed_data = hashlib.sha256(str(data).encode()).hexdigest()
                with open('data_secure.txt', 'a') as f:
                    f.write(hashed_data + '\n')  # Storing hashed data
                self.send_response(200)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"message": "Data received securely."}).encode())
            except Exception as e:
                self.send_response(400)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"error": str(e)}).encode())

if __name__ == '__main__':
    server = HTTPServer(('localhost', 8000), SecureHandler)
    print("Secure server running on port 8000")
    server.serve_forever()
```

Output:-

```
Secure server running on port 8000
```

Justification:
The improved version strengthens the application by addressing critical vulnerabilities such as lack of input validation, insecure data storage, and missing authentication mechanisms. These flaws can lead to attacks like SQL injection, data breaches, and unauthorized access. By applying secure coding practices, the application becomes more resilient, ensuring data protection and system integrity.

Task 5 – Exception Handling Improvement
Task: Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.
Instructions:
• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
• Identify possible runtime errors (e.g., invalid input, division by zero).
• Modify the program to include proper try-except blocks.
• Provide meaningful error messages for invalid inputs.
• Test the improved version with valid and invalid test cases.
Expected Output:
A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:-
Generate a Python program that performs arithmetic operations based on user input. Identify potential runtime errors such as invalid input and division by zero. Improve the program by implementing try-except blocks, using a loop to repeatedly prompt the user until valid input is provided, and displaying clear error messages.

Code:-

```
# Task-05
# Generate a python code wrap the arithmetic logic in a try-except block to catch sp
def safe_division():
    while True:
        try:
            num1 = float(input("Enter the first number: "))
            num2 = float(input("Enter the second number: "))
            result = num1 / num2
            print(f"The result of {num1} divided by {num2} is: {result}")
            break  # Exit the loop if the operation is successful
        except ValueError:
            print("Invalid input. Please enter numeric values.")
        except ZeroDivisionError:
            print("Cannot divide by zero. Please enter a non-zero second number.")
if __name__ == "__main__":
    safe_division()
```

Output:-

```
Enter the first number: 50
Enter the second number: 90
The result of 50.0 divided by 90.0 is: 0.5555555555555556
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The program highlights the security risks of storing passwords in plain text and demonstrates how to mitigate these risks using hashing techniques. Plain-text storage makes sensitive user data highly vulnerable to breaches and unauthorized access. By implementing hashing, passwords are transformed into irreversible formats, ensuring better protection even if data is compromised. Additionally, generating strong passwords enhances resistance against brute-force attacks. Overall, the improved approach ensures secure authentication while maintaining functionality.